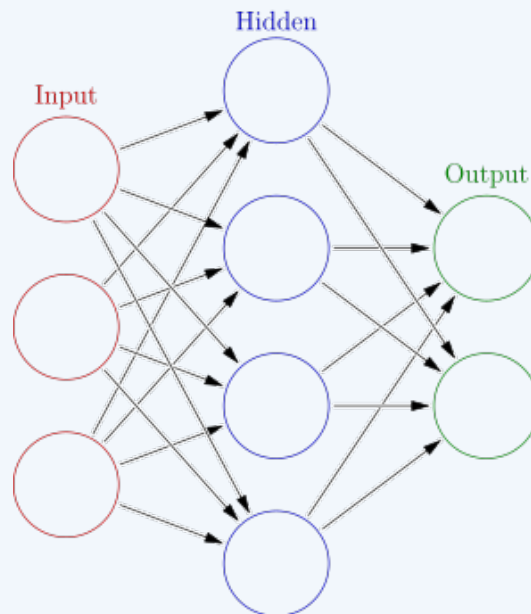


RAPPORT DE STAGE

Visualisation d'un Réseau de Neurones



Réalisé par : Diallo Moussa

Encadré par : Loïc Barthe

Année Universitaire : 2024-2025

Table des matières

1	Introduction	3
1.1	Problématique	3
1.2	Objectifs du projet	3
1.3	Plan du rapport	3
2	Fondements Théoriques des Réseaux de Neurones	5
2.1	Qu'est-ce qu'un Réseau de Neurones ?	5
2.1.1	Le neurone biologique vs. artificiel	5
2.2	Structure d'un Réseau de Neurones	5
2.2.1	Les couches du réseau	5
2.3	Fonctions d'Activation	6
2.3.1	Fonction Sigmoidé	6
2.3.2	Fonction ReLU	6
2.3.3	Fonction Tanh	6
3	Le Perceptron Simple	8
3.1	Problématique et Modélisation	8
3.2	Définition Mathématique du Perceptron	8
3.2.1	Modèle mathématique	8
3.2.2	Classification binaire	8
3.3	Fonction de Perte et Optimisation	8
3.3.1	Erreur Quadratique Moyenne (MSE)	8
3.3.2	Descente de Gradient	9
3.3.3	Calcul des gradients	9
3.4	Implémentation de la classe Perceptron	9
3.5	Convergence de l'Algorithme Perceptron	10
3.5.1	Cas des données linéairement séparables	10
3.5.2	Cas des données non linéairement séparables	11
3.5.3	Analyse des résultats	11
3.5.4	Limitation théorique	11
3.5.5	Analyse quantitative de la convergence	12
3.6	Application : Reconstruction d'Image	12
3.6.1	Préparation des données	12
3.6.2	Résultat de la reconstruction	13
4	Réseaux de Neurones Multicouches (MLP)	14
4.1	Architecture et Propagation Avant	14
4.1.1	Propagation avant (Forward Pass)	14
4.2	Rétropropagation (Backpropagation)	14
4.2.1	Calcul des gradients	15
4.2.2	Mise à jour des paramètres	15
4.3	Classification avec MLP	15
4.3.1	Datasets d'expérimentation	15
4.3.2	Implémentation et résultats	15
4.4	Visualisation des frontières de décision	16

5 Réseaux de Neurones Convolutionnels (CNN)	17
5.1 Introduction aux CNN	17
5.1.1 Opération de Convolution	17
5.1.2 Architecture typique d'un CNN	17
5.2 Couches Convolutionnelles	17
5.2.1 Paramètres d'une couche convolutionnelle	17
5.2.2 Calcul de la taille de sortie	17
5.3 Application : Reconstruction d'Image avec CNN	18
5.3.1 Implémentation d'un CNN simple	18
5.3.2 Résultats visuels	18
Conclusion	20

Chapitre 1

Introduction

L'exploration du fonctionnement interne des réseaux de neurones constitue un défi passionnant, se situant au carrefour de plusieurs disciplines clés : l'intelligence artificielle, l'apprentissage automatique et la visualisation des données. Ces modèles informatiques, dont l'architecture est inspirée par la complexité du cerveau humain, possèdent une capacité remarquable à apprendre de manière autonome à partir de vastes ensembles de données brutes. Grâce à cette faculté d'apprentissage, ils excellent dans l'accomplissement de tâches complexes, allant de la reconnaissance d'images à la traduction automatique de textes, en passant par la prédiction de comportements futurs avec une précision souvent surprenante. Leur omniprésence dans le paysage technologique moderne témoigne de leur efficacité et de leur potentiel transformateur.

Cependant, malgré leurs performances impressionnantes et leur intégration croissante dans de multiples applications, les réseaux de neurones conservent une nature intrinsèquement complexe qui les rend souvent opaques, tel un labyrinthe interne difficile à déchiffrer. Ils sont fréquemment décrits comme des "boîtes noires" : nous pouvons observer et évaluer avec précision les résultats qu'ils produisent, mais la compréhension détaillée des mécanismes internes par lesquels ces résultats sont obtenus reste un défi majeur. Cette opacité n'est pas sans conséquence ; elle représente un obstacle significatif à leur adoption pleine et entière, en particulier dans des domaines critiques où la transparence, la traçabilité et l'explicabilité des décisions prises par le système sont non seulement souhaitables, mais absolument primordiales. Comprendre pourquoi un réseau de neurones a pris une décision spécifique est souvent aussi important, sinon plus, que la décision elle-même, surtout lorsqu'il s'agit d'applications impactant directement la vie humaine ou des processus de haute valeur.

1.1 Problématique

Comment peut-on visualiser et comprendre les transformations successives que subissent les données à travers les différentes couches d'un réseau de neurones ? Cette question fondamentale motive le développement d'outils pédagogiques permettant de démystifier ces systèmes complexes.

1.2 Objectifs du projet

L'objectif principal de ce projet est de développer un outil de visualisation interactif capable d'illustrer :

- Les transformations successives des données dans un réseau de neurones
- L'effet des fonctions d'activation sur les données
- La convergence de l'algorithme d'apprentissage
- L'influence des hyperparamètres sur les performances

1.3 Plan du rapport

Ce rapport s'articule autour de plusieurs chapitres :

- Le chapitre 2 présente les fondements théoriques des réseaux de neurones

- Le chapitre 3 détaille l'implémentation du perceptron simple
- Le chapitre 4 explore les réseaux de neurones multicouches
- Le chapitre 5 introduit les réseaux de neurones convolutionnels
- Le chapitre 6 présente les résultats expérimentaux
- Le chapitre 7 conclut et propose des perspectives

Chapitre 2

Fondements Théoriques des Réseaux de Neurones

2.1 Qu'est-ce qu'un Réseau de Neurones ?

Un réseau de neurones est un modèle computationnel inspiré du fonctionnement des neurones biologiques dans le cerveau humain. Ces modèles sont largement utilisés en apprentissage automatique pour modéliser des relations complexes entre variables d'entrée et de sortie.

2.1.1 Le neurone biologique vs. artificiel

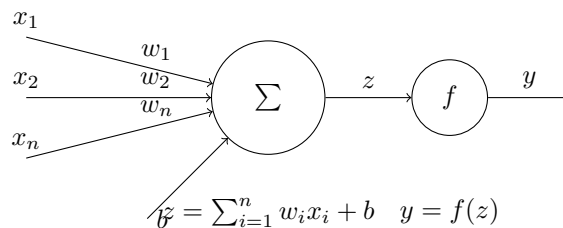


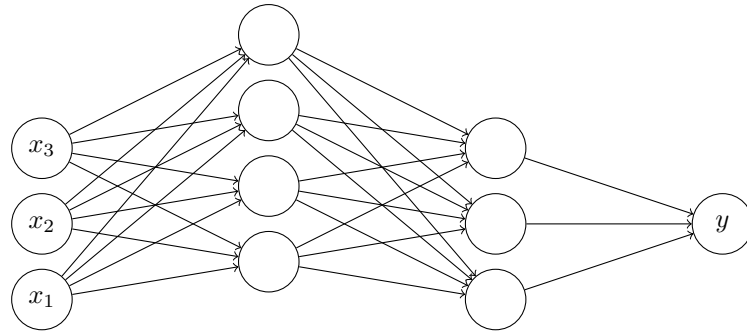
FIGURE 2.1 – Structure d'un neurone artificiel

2.2 Structure d'un Réseau de Neurones

2.2.1 Les couches du réseau

Un réseau de neurones typique est organisé en trois types de couches :

1. **Couche d'entrée** : Reçoit les données d'entrée. Chaque nœud correspond à une caractéristique.
2. **Couches cachées** : Effectuent le traitement principal des données.
3. **Couche de sortie** : Produit le résultat final (classification ou régression).



Couche d'entrée Couche cachée 1 Couche cachée 2 Couche de sortie

FIGURE 2.2 – Architecture d'un réseau de neurones multicouche

2.3 Fonctions d'Activation

Les fonctions d'activation introduisent la non-linéarité dans le réseau, permettant d'approximer des fonctions complexes.

2.3.1 Fonction Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.1)$$

Propriétés :

- Sortie dans l'intervalle $(0, 1)$
- Dérivée : $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Problème du gradient qui disparaît

2.3.2 Fonction ReLU

$$\text{ReLU}(z) = \max(0, z) \quad (2.2)$$

Propriétés :

- Simple à calculer
- Dérivée : $\text{ReLU}'(z) = \begin{cases} 1 & \text{si } z > 0 \\ 0 & \text{si } z \leq 0 \end{cases}$
- Résout partiellement le problème du gradient qui disparaît

2.3.3 Fonction Tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (2.3)$$

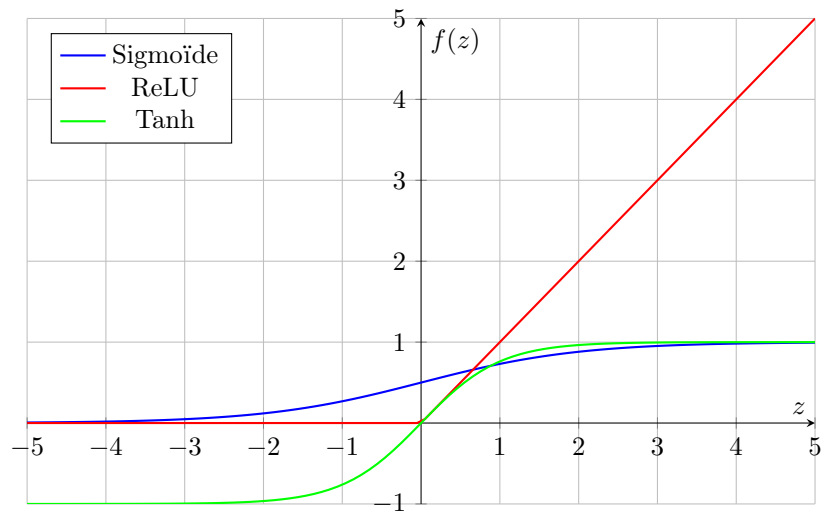


FIGURE 2.3 – Comparaison des fonctions d'activation

Chapitre 3

Le Perceptron Simple

3.1 Problématique et Modélisation

Considérons une image en niveaux de gris de taille $N \times N$. Chaque pixel est identifié par ses coordonnées (x, y) et possède une valeur C (niveau de gris). On cherche à approximer la fonction :

$$f : \mathbb{R}^2 \rightarrow \mathbb{R} \text{ telle que } f(x, y) = C \quad (3.1)$$

où C est la valeur du pixel situé en (x, y) .

3.2 Définition Mathématique du Perceptron

Le perceptron est un modèle de neurone artificiel qui effectue une combinaison linéaire de ses entrées, suivie d'une fonction d'activation.

3.2.1 Modèle mathématique

Pour une entrée $\mathbf{x} \in \mathbb{R}^n$, la sortie du perceptron est :

$$\hat{y} = f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b \quad (3.2)$$

où :

- $\mathbf{w} \in \mathbb{R}^n$ est le vecteur de poids
- $b \in \mathbb{R}$ est le biais

3.2.2 Classification binaire

Pour la classification binaire, on applique une fonction de seuil :

$$\text{sortie} = \text{sign}(\mathbf{w}^T \mathbf{x} + b) \quad (3.3)$$

3.3 Fonction de Perte et Optimisation

3.3.1 Erreur Quadratique Moyenne (MSE)

Pour la régression, nous utilisons la fonction de perte MSE :

$$L(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (f(\mathbf{x}^{(i)}) - y^{(i)})^2 \quad (3.4)$$

3.3.2 Descente de Gradient

Pour minimiser la perte, nous mettons à jour les paramètres selon :

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}} \quad (3.5)$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b} \quad (3.6)$$

où η est le taux d'apprentissage.

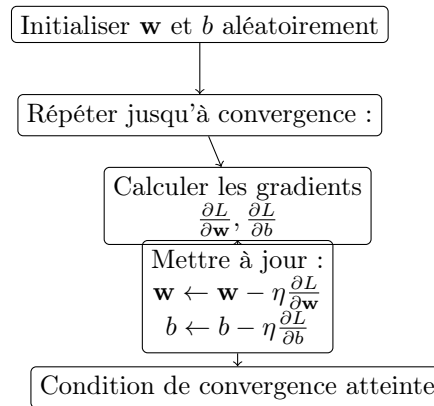


FIGURE 3.1 – Algorithme de la descente de gradient

3.3.3 Calcul des gradients

$$\frac{\partial L}{\partial \mathbf{w}} = \frac{2}{N} \sum_{i=1}^N (f(\mathbf{x}^{(i)}) - y^{(i)}) \mathbf{x}^{(i)} \quad (3.7)$$

$$\frac{\partial L}{\partial b} = \frac{2}{N} \sum_{i=1}^N (f(\mathbf{x}^{(i)}) - y^{(i)}) \quad (3.8)$$

3.4 Implémentation de la classe Perceptron

Nous utilisons notre implémentation de la classe Perceptron qui enregistre l'historique des mises à jour :

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class Perceptron:
5     def __init__(self, add_bias=True, max_iters=10000, record_updates=False):
6         self.max_iters = max_iters
7         self.add_bias = add_bias
8         self.record_updates = record_updates
9         if record_updates:
10             self.w_hist = [] # enregistre les poids
11             self.n_hist = [] # enregistre l'indice du point selectionne
12
13     def fit(self, x, y):
14         if x.ndim == 1:
15             x = x[:, None]
16         if self.add_bias:
17             N = x.shape[0]
18             x = np.column_stack([x, np.ones(N)]) # ajout du biais
19         N, D = x.shape
20         w = np.zeros(D) # initialisation des poids z ro
21         # Conversion 0,1 vers -1,+1
  
```

```

22     y = 2 * y - 1
23     t = 0
24     change = True
25     while change and t < self.max_iters:
26         change = False
27         for n in np.random.permutation(N):
28             yh = np.sign(np.dot(x[n, :], w)) # pr diction
29             if yh == y[n]:
30                 continue # point bien class
31             w = w + y[n] * x[n, :] # mise jour des poids
32
33             if self.record_updates:
34                 self.w_hist.append(w.copy())
35                 self.n_hist.append(n)
36             change = True
37         t += 1
38     self.w = w
39     return self
40
41     def predict(self, x):
42         if x.ndim == 1:
43             x = x[:, None]
44         if self.add_bias:
45             x = np.column_stack([x, np.ones(x.shape[0])])
46         yh = np.sign(np.dot(x, self.w))
47         return (yh + 1) // 2 # conversion -1/+1 vers 0/1

```

Listing 3.1 – Classe Perceptron avec enregistrement des mises à jour

3.5 Convergence de l’Algorithme Perceptron

3.5.1 Cas des données linéairement séparables

Pour commencer, nous testons l’algorithme du perceptron sur un cas où les données sont linéairement séparables en utilisant les deux premières classes du dataset Iris.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn import datasets
4
5 # Chargement du dataset Iris (2 classes, 2 features)
6 dataset = datasets.load_iris()
7 x, y = dataset['data'][:, 2:], dataset['target'] # les 2 dernières features
8 x, y = x[y < 2, :], y[y < 2] # classes 0 et 1 uniquement
9
10 # On suppose que ta classe Perceptron s'appelle bien Perceptron et poss de w_hist
11 model = Perceptron(record_updates=True)
12 model.fit(x, y)
13
14 # Trac des données avec couleurs modifiées
15 plt.plot(x[y == 0, 0], x[y == 0, 1], 'k.', label='Classe 0') # points classe 0 en
    noir
16 plt.plot(x[y == 1, 0], x[y == 1, 1], 'r.', label='Classe 1') # points classe 1 en
    rouge
17
18 x_line = np.linspace(np.min(x[:, 0]), np.max(x[:, 0]), 100)
19
20 # Trac de la frontière en vert, alpha variant
21 for t, w in enumerate(model.w_hist):
22     coef = -w[0] / w[1] # pente
23     intercept = -w[2] / w[1] # intercept
24     plt.plot(x_line, coef * x_line + intercept, color='green',
25              alpha=t / len(model.w_hist), label=f't={t}' if t != 0 else None)
26
27 plt.xlabel(r'$x_1$')
28 plt.ylabel(r'$x_2$')
29 plt.title('Convergence de l\'algorithme')
30 plt.legend()
31 plt.show()

```

Listing 3.2 – Test de convergence sur données linéairement séparables

3.5.2 Cas des données non linéairement séparables

Maintenant, nous testons l'algorithme sur un cas plus complexe où les données ne sont pas linéairement séparables.

```

1 # Chargement du dataset Iris avec features différentes
2 dataset = datasets.load_iris()
3 x, y = dataset['data'][:,1:2], dataset['target'] # slice feature 1 and 2 of Iris
4 dataset
5 y = y > 1
6 model = Perceptron(record_updates=True)
7 yh = model.fit(x, y)
8
9 plt.plot(x[y==0,0], x[y==0,1], 'k.' )
10 plt.plot(x[y==1,0], x[y==1,1], 'b.' )
11
12 x_line = np.linspace(np.min(x[:,0]), np.max(x[:,0]), 100)
13 for t,w in enumerate(model.w_hist):
14     coef = -w[0]/w[1]
15     plt.plot(x_line, coef*x_line - w[2]/w[1], 'r-', alpha=t/len(model.w_hist), label=f
16             't={t}')
17 plt.xlabel(r'$x_1$')
18 plt.ylabel(r'$x_2$')
19 plt.ylim(-1,10)
20 plt.title('Perceptron quand le data n est pas lin airement s parable')
21 plt.show()

```

Listing 3.3 – Test de convergence sur données non linéairement séparables

3.5.3 Analyse des résultats

Nous Visualisons la progression de l'algorithme Perceptron au fil des mises à jour des poids.

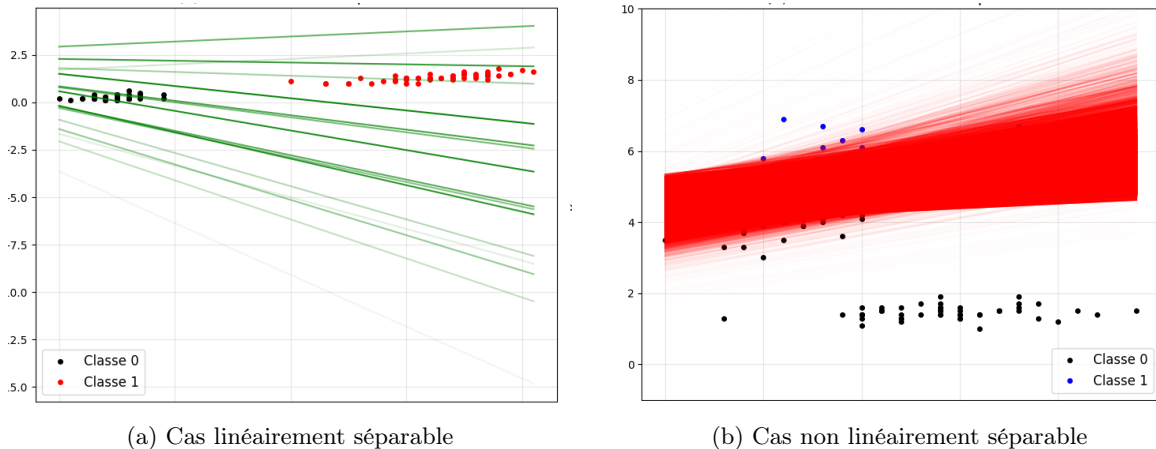


FIGURE 3.2 – Comparaison du comportement du perceptron : (a) convergence rapide sur données séparables, (b) oscillation continue sur données non séparables.

3.5.4 Limitation théorique

Le théorème de convergence du perceptron stipule que :

Théorème 1 (Convergence du Perceptron). *Si un ensemble de données est linéairement séparable, alors l'algorithme du perceptron converge en un nombre fini d'itérations vers une solution qui sépare parfaitement les classes.*

Corollaire : Si les données ne sont pas linéairement séparables, l'algorithme ne converge pas et oscille indéfiniment.

3.5.5 Analyse quantitative de la convergence

Pour mieux comprendre le comportement de l'algorithme, nous pouvons analyser l'évolution du taux d'erreur :

```
1 # Calcul de l'erreur de classification chaque mise jour
2 errors = []
3 for i, w in enumerate(model.w_hist):
4     # Pr dictions avec les poids actuels
5     predictions = np.sign(np.dot(x, w[:-1]) + w[-1])
6     predictions = (predictions + 1) // 2 # Conversion -1/+1 vers 0/1
7
8     # Calcul de l'erreur
9     error_rate = np.mean(predictions != y)
10    errors.append(error_rate)
11
12 # Trac de l'volution de l'erreur
13 plt.figure(figsize=(10, 6))
14 plt.plot(errors, 'b-', linewidth=2)
15 plt.xlabel('It ration')
16 plt.ylabel('Taux d'erreur')
17 plt.title('volution du taux d'erreur durant l'apprentissage')
18 plt.grid(True, alpha=0.3)
19 plt.axhline(y=min(errors), color='r', linestyle='--',
20             label=f'Erreur minimale: {min(errors):.3f}')
21 plt.legend()
22 plt.show()
23
24 print(f"Nombre total de mises jour: {len(model.w_hist)}")
25 print(f"Erreur finale: {errors[-1]:.3f}")
26 print(f"Erreur minimale atteinte: {min(errors):.3f}")
```

Listing 3.4 – Analyse de l'évolution de l'erreur

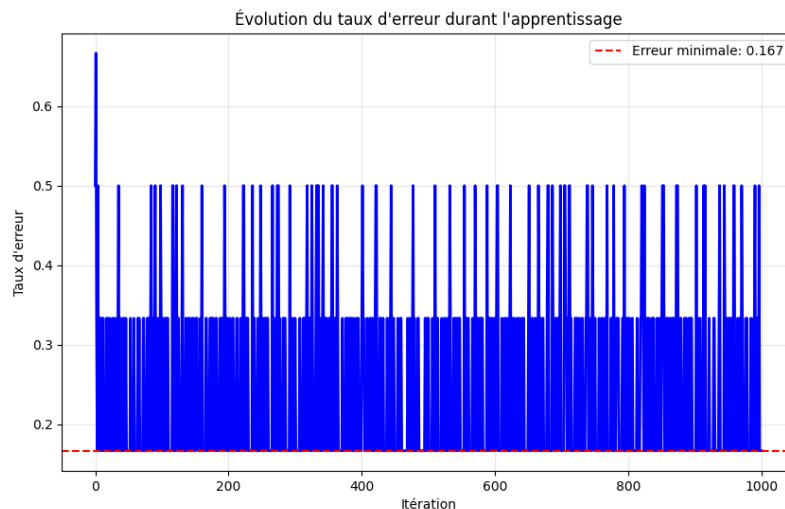


FIGURE 3.3 – Évolution du taux d'erreur durant l'apprentissage. L'oscillation de l'erreur montre que l'algorithme ne converge pas vers une solution stable sur des données non linéairement séparables.

Cette expérimentation illustre clairement la limitation fondamentale du perceptron simple : il ne peut résoudre que des problèmes linéairement séparables, ce qui motive l'introduction des réseaux multicouches traités au chapitre suivant.

3.6 Application : Reconstruction d'Image

3.6.1 Préparation des données

```
1 # Charger une image en niveaux de gris
2 img = Image.open(img_path).convert('L').resize((128, 128))
```

```

3 img_np = np.array(img) / 255.0 # Normalisation entre 0 et 1
4 h, w = img_np.shape
5
6 # Préparer les coordonnées comme entrées
7 x, y = np.meshgrid(np.arange(w), np.arange(h))
8 X = np.stack([x.ravel(), y.ravel()], axis=1) / [w, h] # Normalisation
9 Y = img_np.ravel()[ :, None] # Valeurs des pixels
10
11 # Définir le modèle
12 model = tf.keras.Sequential([
13     tf.keras.layers.Dense(64, activation='relu', input_shape=(2,)),
14     tf.keras.layers.Dense(64, activation='relu'),
15     tf.keras.layers.Dense(1, activation='sigmoid')
16 ])
17
18 model.compile(optimizer='adam', loss='mse')
19 model.fit(X, Y, epochs=50, batch_size=1024, verbose=1)
20
21 # Prédiction et reconstruction
22 Y_pred = model.predict(X).reshape(h, w)

```

Listing 3.5 – Préparation des données pour la reconstruction d'image

3.6.2 Résultat de la reconstruction

```

1 # Afficher l'image originale et reconstruite
2 plt.subplot(1,2,1)
3 plt.imshow(img_np, cmap='gray')
4 plt.axis('off')
5 plt.title("Originale")
6
7 plt.subplot(1,2,2)
8 plt.imshow(Y_pred, cmap='gray')
9 plt.axis('off')
10 plt.title("Reconstruction")
11
12 plt.tight_layout()
13 plt.show()

```

Listing 3.6 – Affichage des images originale et reconstruite

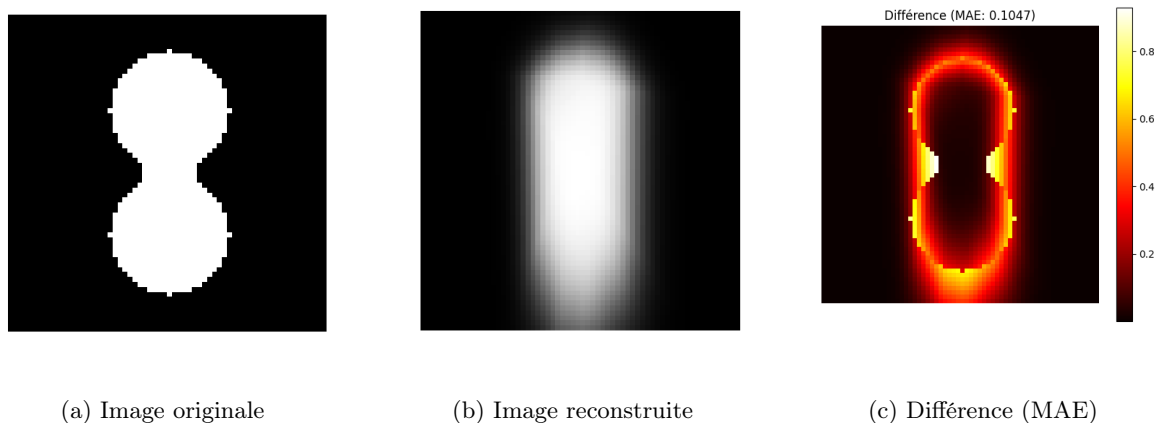


FIGURE 3.4 – (a) Image originale, (b) image reconstruite, (c) différence (MAE) après entraînement du réseau.

Chapitre 4

Réseaux de Neurones Multicouches (MLP)

4.1 Architecture et Propagation Avant

Un réseau multicouche (Multilayer Perceptron - MLP) étend le perceptron simple en ajoutant des couches cachées.

4.1.1 Propagation avant (Forward Pass)

Pour un réseau à L couches :

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \quad (4.1)$$

$$\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad (4.2)$$

où :

- $\mathbf{a}^{(0)} = \mathbf{x}$ (entrée)
- $\mathbf{W}^{(l)}$: matrice des poids de la couche l
- $\mathbf{b}^{(l)}$: vecteur de biais de la couche l
- $f^{(l)}$: fonction d'activation de la couche l

Configuration

- Architecture : 4-5-3-1
- Activation : ReLU (cachées)
- Activation : Sigmoidé (sortie)

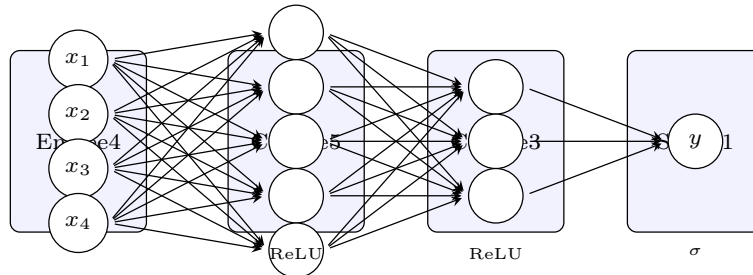


FIGURE 4.1 – Réseau de neurones multicouche 4-5-3-1 avec ReLU et Sigmoidé

4.2 Rétropropagation (Backpropagation)

L'algorithme de rétropropagation calcule les gradients de la fonction de perte par rapport aux paramètres.

4.2.1 Calcul des gradients

Pour la couche de sortie ($l = L$) :

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} L \odot f'^{(L)}(\mathbf{z}^{(L)}) \quad (4.3)$$

Pour les couches cachées ($l = L - 1, \dots, 1$) :

$$\delta^{(l)} = ((\mathbf{W}^{(l+1)})^T \delta^{(l+1)}) \odot f'^{(l)}(\mathbf{z}^{(l)}) \quad (4.4)$$

4.2.2 Mise à jour des paramètres

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T \quad (4.5)$$

$$\frac{\partial L}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \quad (4.6)$$

4.3 Classification avec MLP

4.3.1 Datasets d'expérimentation

Nous utilisons plusieurs datasets pour tester les performances :

1. **Make Moons** : Dataset non-linéairement séparable
2. **Make Circles** : Classification circulaire
3. **Linearly Separable** : Dataset linéairement séparable

4.3.2 Implémentation et résultats

```
1 from sklearn.neural_network import MLPClassifier
2 from sklearn.preprocessing import StandardScaler
3 from sklearn.pipeline import make_pipeline
4
5 # Configuration des hyperparam tres
6 alphas = [0.00005, 0.002, 0.025] # Regularisation L2
7
8 classifiers = []
9 for alpha in alphas:
10     clf = make_pipeline(
11         StandardScaler(),
12         MLPClassifier(
13             solver="lbfgs",
14             alpha=alpha,
15             random_state=1,
16             max_iter=2000,
17             early_stopping=True,
18             hidden_layer_sizes=[12, 8]
19         )
20     )
21     classifiers.append(clf)
22
23 # Entra nement et valuation
24 for X, y in datasets:
25     X_train, X_test, y_train, y_test = train_test_split(
26         X, y, test_size=0.4, random_state=42
27     )
28
29     for clf in classifiers:
30         clf.fit(X_train, y_train)
31         score = clf.score(X_test, y_test)
32         print(f"Accuracy: {score:.3f}")
```

Listing 4.1 – Configuration du MLP pour la classification

4.4 Visualisation des frontières de décision

Pour chaque dataset, nous avons testé trois valeurs du paramètre de régularisation α et visualisé la frontière de décision obtenue.

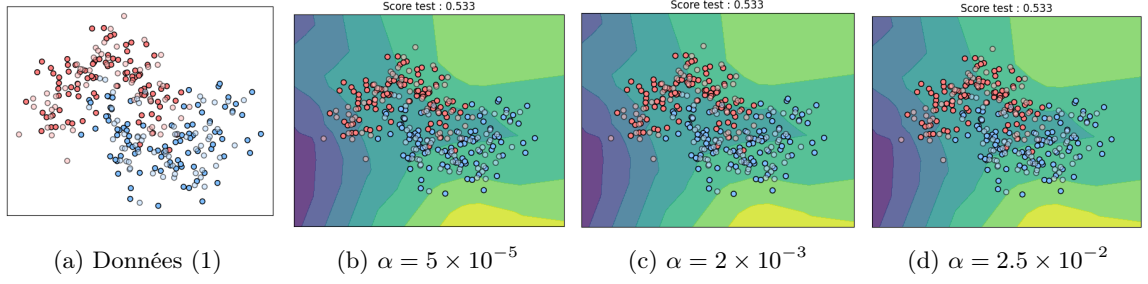


FIGURE 4.2 – Résultats pour le dataset 1 (*Make Moons*) : (a) données, (b-d) frontières de décision pour différentes valeurs de α .

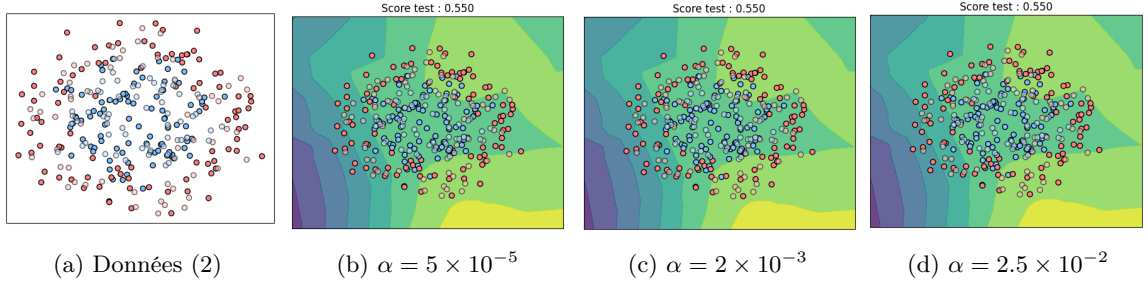


FIGURE 4.3 – Résultats pour le dataset 2 (*Make Circles*) : (a) données, (b-d) frontières de décision pour différentes valeurs de α .

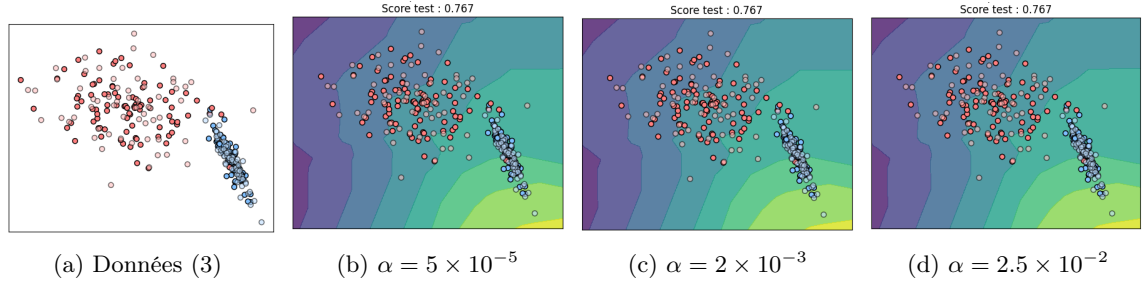


FIGURE 4.4 – Résultats pour le dataset 3 (linéairement séparable) : (a) données, (b-d) frontières de décision pour différentes valeurs de α .

Chapitre 5

Réseaux de Neurones Convolutionnels (CNN)

5.1 Introduction aux CNN

Les réseaux de neurones convolutionnels (CNN) sont spécialement conçus pour traiter des données ayant une structure de grille, comme les images.

5.1.1 Opération de Convolution

L'opération de convolution 2D est définie par :

$$(I * K)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (5.1)$$

où I est l'image d'entrée et K est le noyau (kernel).

5.1.2 Architecture typique d'un CNN

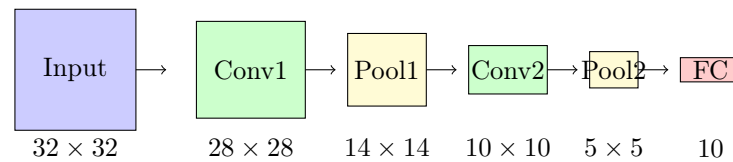


FIGURE 5.1 – Architecture typique d'un CNN

5.2 Couches Convolutionnelles

5.2.1 Paramètres d'une couche convolutionnelle

- **Nombre de filtres** : Détermine la profondeur de la sortie
- **Taille du filtre** : Généralement 3×3 ou 5×5
- **Stride** : Pas de déplacement du filtre
- **Padding** : Ajout de zéros sur les bords

5.2.2 Calcul de la taille de sortie

Pour une entrée de taille (H_{in}, W_{in}) avec un filtre de taille (F, F) , stride S et padding P :

$$H_{out} = \frac{H_{in} + 2P - F}{S} + 1 \quad (5.2)$$

$$W_{out} = \frac{W_{in} + 2P - F}{S} + 1 \quad (5.3)$$

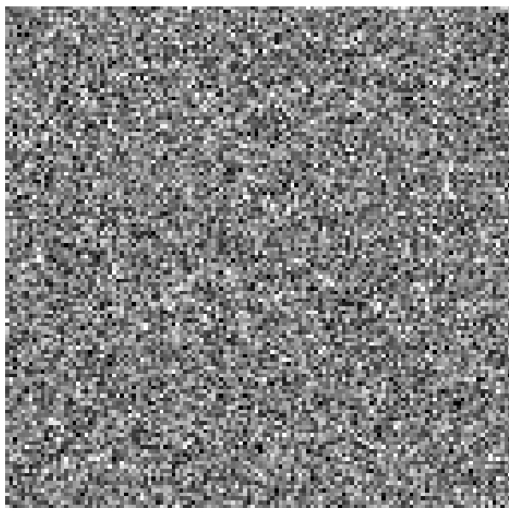
5.3 Application : Reconstruction d'Image avec CNN

5.3.1 Implémentation d'un CNN simple

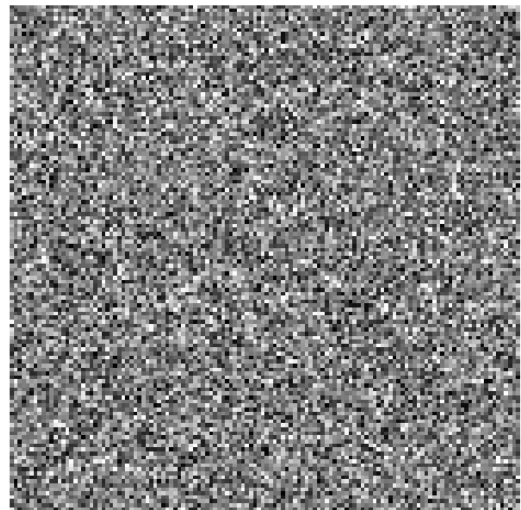
```
1 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
2
3 # Définition du modèle CNN
4 model = tf.keras.Sequential([
5     # Couche d'entrée reshape pour convertir les coordonnées en "image"
6     tf.keras.layers.Reshape((128, 128, 1), input_shape=(128, 128)),
7
8     # Première couche convolutionnelle
9     Conv2D(32, (3, 3), activation='relu', padding='same'),
10    MaxPooling2D((2, 2)),
11
12    # Deuxième couche convolutionnelle
13    Conv2D(64, (3, 3), activation='relu', padding='same'),
14    MaxPooling2D((2, 2)),
15
16    # Couches fully connected
17    Flatten(),
18    Dense(128, activation='relu'),
19    Dense(64, activation='relu'),
20    Dense(1, activation='sigmoid')
21 ])
22
23 model.compile(optimizer='adam', loss='mse')
24
25 # Préparation des données
26 X = np.stack([x_coords, y_coords], axis=-1) # Coordonnées normalisées
27 Y = img_np.ravel() # Valeurs des pixels
28
29 model.fit(X, Y, epochs=30, batch_size=1024)
30
31 # Prédiction
32 Y_pred = model.predict(X).reshape(128, 128)
```

Listing 5.1 – Implémentation d'un CNN pour la reconstruction d'image

5.3.2 Résultats visuels



(a) Image originale



(b) Image reconstruite

FIGURE 5.2 – Comparaison entre l'image d'entrée et sa reconstruction par le CNN.

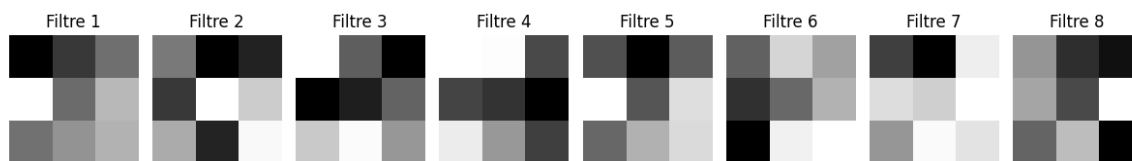


FIGURE 5.3 – Filtres appris par la première couche convolutionnelle ('conv1').

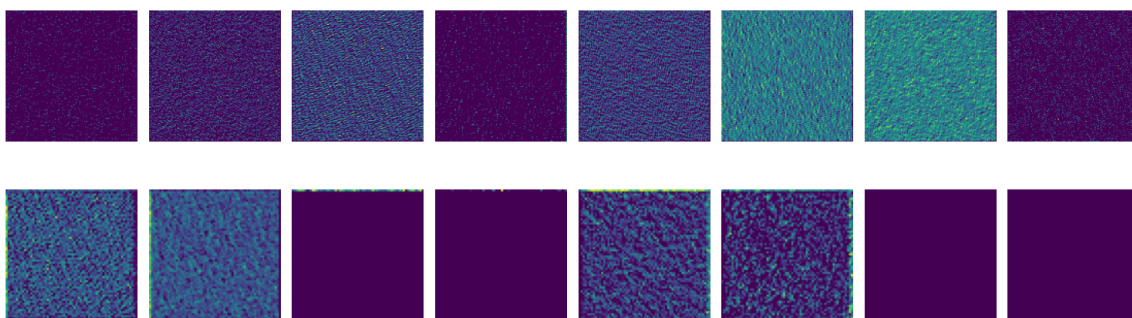


FIGURE 5.4 – Activations des couches 'conv1' et 'conv2' pour l'image d'entrée.

Conclusion

Ce rapport de stage a exploré les réseaux de neurones, des modèles d'apprentissage automatique inspirés du cerveau. Nous avons d'abord compris les bases, du neurone simple aux architectures multicouches, et l'importance des fonctions d'activation pour la non-linéarité.

Le Perceptron simple nous a montré ses limites avec les problèmes non-linéaires, ce qui a justifié l'utilisation de réseaux multicouches (MLP) capables de gérer des relations plus complexes grâce à la rétropropagation. Enfin, nous avons abordé les réseaux de neurones convolutionnels (CNN), particulièrement efficaces pour le traitement d'images. Ce stage a renforcé ma compréhension théorique et pratique de ces outils puissants, en soulignant l'importance de la visualisation pour démystifier leur fonctionnement interne.

Bibliographie

- [1] Ian Goodfellow, Yoshua Bengio et Aaron Courville, *Deep Learning*, MIT Press, 2016.
- [2] Frank Rosenblatt, « The Perceptron : A Probabilistic Model For Information Storage And Organization In The Brain », *Psychological Review*, vol. 65, n° 6, pp. 386–408, 1958.
- [3] Yann LeCun, Léon Bottou, Yoshua Bengio et Patrick Haffner, « Gradient-Based Learning Applied to Document Recognition », *Proceedings of the IEEE*, vol. 86, n° 11, pp. 2278–2324, 1998.
- [4] F. Pedregosa et al., « Scikit-learn : Machine Learning in Python », *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] Charles R. Harris et al., « Array programming with NumPy », *Nature*, vol. 585, pp. 357–362, 2020.
- [6] J. D. Hunter, « Matplotlib : A 2D graphics environment », *Computing in Science & Engineering*, vol. 9, n° 3, pp. 90–95, 2007.
- [7] François Chollet, *Keras*, GitHub repository, 2015, <https://github.com/keras-team/keras>.
- [8] Martín Abadi et al., « TensorFlow : Large-Scale Machine Learning on Heterogeneous Systems », arXiv :1603.04467, 2016.
- [9] Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.